

Frontend Interview Handbook — Part 6: APIs, React Query, State Management & Security

SECTION A: REST APIs & DATA FETCHING

Q1: What makes an API "RESTful"? Explain the core principles.

Answer: REST (Representational State Transfer) is an architectural style built on: **statelessness** (each request contains everything needed; server holds no client session state between requests), **resource-based URLs** (`/users/123`, not `/getUserById?id=123`), **standard HTTP methods** mapped to actions (GET read, POST create, PUT/PATCH update, DELETE remove), and **uniform representations** (typically JSON) with proper status codes communicating outcome. Since you've built DRF APIs yourself, this is a strength — be ready to speak from direct implementation experience, not just definitions.

Q2: What's the difference between PUT and PATCH?

Answer: `PUT` replaces the entire resource — you send the complete representation, and any omitted fields are typically reset/removed. `PATCH` applies a partial update — you send only the fields that changed, and the rest of the resource stays untouched. A common mistake is using `PUT` for partial updates, which can unintentionally wipe fields the client didn't include.

Q3: Explain key HTTP status codes and what each communicates: 200, 201, 204, 301/302, 400, 401, 403, 404, 429, 500.

Answer:

- **200 OK** — success, response body included.
- **201 Created** — resource successfully created (typical POST response).
- **204 No Content** — success, nothing to return (typical DELETE response).
- **301/302** — permanent/temporary redirect.
- **400 Bad Request** — malformed request (client error, e.g., failed validation).
- **401 Unauthorized** — not authenticated (missing/invalid credentials).
- **403 Forbidden** — authenticated but not permitted (authorization failure).
- **404 Not Found** — resource doesn't exist.
- **429 Too Many Requests** — rate limit exceeded.
- **500 Internal Server Error** — unhandled server-side failure.

Follow-up: "What's the practical difference between handling a 401 vs a 403 in the frontend?" A: 401 typically means redirect to login/refresh the auth token; 403 means the

user is logged in but shouldn't see this — usually show a permission-denied state, not a login prompt.

Q4: How do you handle authentication in a React app talking to a token-based backend (e.g., Django REST Framework)?

Answer: Store the token in an `httpOnly` cookie where possible (safer against XSS than `localStorage`, since JS can't read it). Attach it automatically via cookie, or manually via an `Authorization: Bearer <token>` header if using `localStorage`/memory. Use a fetch/axios interceptor to catch 401 responses globally — either refresh the token silently (if using refresh tokens) or redirect to login. Keep auth state in a context or top-level store so the whole app can react to login/logout consistently.

```
js
axios.interceptors.response.use(
  res => res,
  async (error) => {
    if (error.response?.status === 401) {
      // attempt token refresh or redirect to login
    }
    return Promise.reject(error);
  }
);
```

Q5: What is CORS, and why does it exist?

Answer: CORS (Cross-Origin Resource Sharing) is a browser security mechanism that blocks a webpage from making requests to a different origin (domain/port/protocol) than the one it was served from, unless the server explicitly allows it via response headers (`Access-Control-Allow-Origin`, etc.). It exists to prevent malicious sites from silently making authenticated requests to other sites on a user's behalf. As a frontend dev integrating with Django, you'll commonly hit this in local dev (frontend on `localhost:3000`, API on `localhost:8000`) and need the backend to explicitly allow the frontend's origin — good to mention you've configured this in DRF (`django-cors-headers`).

SECTION B: REACT QUERY / DATA FETCHING LIBRARIES

Q6: What problem does React Query (TanStack Query) solve that plain `useEffect` + `fetch` doesn't?

Answer: Plain `useEffect` fetching requires manually handling loading/error states, race conditions (stale requests resolving after a newer one), caching, deduplication of identical

concurrent requests, background refetching, and cache invalidation — all boilerplate you'd otherwise hand-roll and likely get subtly wrong. React Query provides all of this out of the box: automatic caching keyed by query key, background refetch on window refocus/reconnect, built-in loading/error/success states, and mutation handling with optimistic updates.

```
jsx

function useUser(id) {
  return useQuery({
    queryKey: ['user', id],
    queryFn: () => fetch(`/api/users/${id}`).then(r => r.json()),
  });
}

function Profile({ id }) {
  const { data, isLoading, error } = useUser(id);
  if (isLoading) return <Spinner />;
  if (error) return <ErrorMessage error={error} />;
  return <div>{data.name}</div>;
}
```

Q7: What is a "race condition" in data fetching, and how does React Query (or manual code) prevent it?

Answer: If a component fetches based on a changing value (e.g., a search query) and a fast-typing user triggers request A then B, there's no guarantee A resolves before B — if A resolves *after* B, the UI can show stale results from A even though B was the more recent request. React Query handles this automatically via query keys (tracks which request is current). Manually, you'd handle it with a cleanup flag or `AbortController`:

```
js

useEffect(() => {
  const controller = new AbortController();
  fetch(url, { signal: controller.signal }).then(...);
  return () => controller.abort(); // cancel stale request
}, [url]);
```

Q8: What's the difference between `staleTime` and `cacheTime`/`gcTime` in React Query?

Answer: `staleTime` is how long fetched data is considered "fresh" — during this window, React Query won't refetch even if the query is re-triggered (e.g., component remount).

`gcTime` (formerly `cacheTime`) is how long *unused* data stays in the cache before being garbage collected entirely. Data can be stale but still cached (so it displays instantly while a background refetch happens), which is the core of React Query's stale-while-revalidate behavior.

SECTION C: STATE MANAGEMENT

Q9: How do you decide between local state, Context, and a global state library (Redux/Zustand)?

Answer: (Framework worth having ready, since you list Redux as a skill.)

- **Local** `useState`/`useReducer`: default for anything only one component (or its direct children via props) needs.
- **Context**: low-frequency, broadly-needed data — theme, authenticated user, locale. Avoid for frequently-changing data since every consumer re-renders on any change unless carefully split/memoized.
- **Redux/Zustand/Jotai**: shared state that's read/written from many distant components, changes frequently, needs middleware (logging, persistence, undo), or benefits from devtools/time-travel debugging.

Common mistake: Reaching for Redux by default for everything — over-engineering simple local UI state into global store slices adds unnecessary complexity.

Q10: What's the difference between Redux and Redux Toolkit (RTK)? Would you use plain Redux today?

Answer: Plain Redux required a lot of boilerplate (hand-written action types, action creators, reducers, manual immutable updates). RTK is the now-standard, official way to write Redux — `createSlice` generates action creators and reducers together, uses Immer internally so you can "mutate" state in reducers safely (it produces an immutable update under the hood), and includes `createAsyncThunk` for async logic and RTK Query for data fetching (a React Query alternative built into the Redux ecosystem). Answer honestly: no reason to hand-write plain Redux today — RTK is the standard.

js

```
const userSlice = createSlice({
  name: 'user',
  initialState: { name: '', loggedIn: false },
  reducers: {
    login(state, action) { state.name = action.payload; state.loggedIn = true;
  },
});
```

Q11: What's an optimistic update, and how would you implement one?

Answer: Instead of waiting for a server response before updating the UI, you update the UI immediately assuming success, then roll back if the request actually fails — makes the app feel instant for common actions (liking a post, toggling a checkbox).

```
jsx

const mutation = useMutation({
  mutationFn: updateTodo,
  onMutate: async (newTodo) => {
    await queryClient.cancelQueries(['todos']);
    const previous = queryClient.getQueryData(['todos']);
    queryClient.setQueryData(['todos'], old => [...old, newTodo]); // optimistic
    return { previous };
  },
  onError: (err, newTodo, context) => {
    queryClient.setQueryData(['todos'], context.previous); // rollback
  },
});
```

SECTION D: SECURITY

Q12: What is XSS (Cross-Site Scripting), and how does React protect against it by default?

Answer: XSS is when an attacker injects malicious script into a page, typically through unsanitized user input rendered as HTML, which then executes in other users' browsers (stealing cookies/tokens, performing actions as them). React escapes all values rendered via JSX by default (`{userInput}` is treated as text, not HTML), which prevents most XSS automatically. The main way to reintroduce the vulnerability is `dangerouslySetInnerHTML` — using it with unsanitized user content is a direct XSS hole, so any use of it should sanitize input first (e.g., with DOMPurify).

Q13: What is CSRF (Cross-Site Request Forgery), and how is it typically prevented?

Answer: CSRF tricks a logged-in user's browser into submitting a request to a site they're authenticated on, without their intent (e.g., an malicious page auto-submitting a form to a bank's transfer endpoint, relying on the browser automatically attaching the user's session cookie). Prevention: CSRF tokens (a unique per-session token the server requires on state-changing requests, which an attacker's page can't know), `SameSite` cookie attribute (restricts when cookies are sent cross-site), and requiring custom headers for AJAX requests (simple cross-site form submissions can't set custom headers). Since you've worked with Django, mention DRF/Django's built-in CSRF token middleware as a concrete example.

Q14: Where should you NOT store sensitive tokens, and why?

Answer: Avoid `localStorage`/`sessionStorage` for auth tokens if possible — any XSS vulnerability anywhere in the app (including a compromised third-party script) can read them directly via JS. `httpOnly` cookies are safer since JS can't access them at all, though they introduce their own CSRF considerations that need to be handled alongside. For highly sensitive apps, short-lived access tokens plus a securely-stored refresh token is the standard pattern.

Q15: What is a Content Security Policy (CSP), and what does it protect against?

Answer: A CSP is a response header that restricts what sources a page is allowed to load scripts, styles, images, etc. from — mitigating XSS impact even if malicious script somehow gets injected, by blocking it from executing if it doesn't match the allowed source policy.

```
Content-Security-Policy: default-src 'self'; script-src 'self'
https://trusted-cdn.com;
```

Rapid-Fire Round

- **Q: What's the difference between authentication and authorization?** A: Authentication = who you are (login); authorization = what you're allowed to do (permissions/roles) — relevant since you built RBAC on SEOP.
- **Q: What's idempotency, and which HTTP methods are idempotent?** A: An idempotent operation produces the same result no matter how many times it's repeated. GET, PUT, DELETE are idempotent; POST is not (each call typically creates a new resource).
- **Q: What's the difference between `fetch` and `axios`?** A: `fetch` is native, requires manual JSON parsing and doesn't reject on HTTP error status (only network failure); `axios` auto-parses JSON, rejects on non-2xx by default, and has built-in interceptor support.

- **Q: What's rate limiting, and where is it typically enforced?** A: Restricting how many requests a client can make in a time window — typically enforced server-side/API-gateway level, though frontend can help by debouncing/throttling requests to avoid hitting limits unnecessarily.
 - **Q: Why avoid storing passwords or secrets in frontend code/environment variables prefixed for client exposure (e.g., `NEXT_PUBLIC_`)?** A: Anything shipped to the client is visible to anyone inspecting the bundle — client-exposed env vars should never contain secrets, only genuinely public config.
-

End of Part 6. Say "next part" for Part 7: Testing, Git, CI/CD & Deployment.